

Lab Assignment 6

Christos Strydis
Rajendra Bishnoi
Max Engelen
Amirreza Movahedin

2026

Contents

1	In-silico brain experiments	2
1.1	Modeling the Inferior-olivary nucleus	2
1.2	The Inferior-Olive challenge	5

Submission guidelines:

- Hand in the lab report in pdf format with the following name:
Lab6.Firstname.Lastname.Studentnumber (for example: Lab6.Pieter_deBoer_123456)
- Hand in the Python scripts (the **.py** files) as separate, unzipped Python scripts. Do not forget to add meaningful comments to your code. Handing in poorly annotated code will be penalized with one point.

1 In-silico brain experiments

In-vivo and in-vitro experiments are a traditional tool of neuroscientific research. They are powerful experimentation methods, but are also time-consuming and not always reliable. A number of factors can contaminate results like, for example, the influence of anesthesia in in-vivo experiments. What is more, most systemic phenomena require the monitoring of biological systems of very large scale and many such techniques do not allow for this kind of study.

Through the efforts of biologists and computational neuroscientists in recent decades, advanced models of cortical neurons were developed using Spiking Neural Networks (SNNs). These models do not capture aspects of biological processes in an abstract manner – like Artificial Neural Networks (ANNs) do – but they straightforwardly emulate them. Such models allow neuroscientists to construct *biologically accurate* models of brain subsystems, furthering their study. SNNs, thus, are widely used in neuroscientific research to complement biological experiments.

1.1 Modeling the Inferior-olivary nucleus

If a researcher seeks to explore the electrochemical characteristics that produce the neuron's response, a biophysically meaningful SNN neuron model is required, such as conductance-based models. These models closely capture the electrochemical behavior that produces the neuron activity by modeling the various ion channels observed inside neurons. The most prominent conductance-based model used today is the one originally presented by Hodgkin and Huxley (HH) in 1952 [2]. **HH models**, and their variants, heavily employ differential equations and are quite scalable, making the design of *multi-compartmental* models possible¹. The Inferior-Olive (IO) model is such a multi-compartmental network model.

The IO model captures part of the **inferior-olivary nucleus**, which is one of the densest brain regions and plays an important role in sensorimotor control. This network can provide rhythm and coordination signals for motor functions. It is considered to be imperative for the instinctive learning and smooth completion of motor actions. It is theorized that enjoying music and dancing would be impossible without the activity from this region of the cerebellum.

What makes the IO neurons special is their dense interconnection through electrical synapses called gap junctions (GJs). These are not to be confused with chemical synapses, which are slower by comparison. Gap junctions facilitate the synchronization behavior between the IO neurons and, subsequently, influence the synchronization and learning properties of the entire olivocerebellar system. The IO nucleus comprises intrinsic *oscillators* that receive converging glutamatergic and gabaergic input from the cerebellum, and the diencephalic junction, respectively; see Figure 1. These inputs encounter the ongoing dynamics of the IO neurons, which they shape and modulate through phase shifts and resets.

The *single-neuron*, IO model was originally developed by De Gruijl et al. [1]. It implements a neuron with three distinct compartments, the dendrite, the soma and the axon. Within the dendrite, the model also includes gap junctions. The dendrites represent the cell input stage, the soma is the cell part wherein most of the neural processing takes place, and the axon represents the cell output stage towards the rest of the cerebellum; see Figure 2(b).

In our ongoing research at the Neuroscience department of the Erasmus Medical Center, Rotterdam, we have constructed a complete, HH-type, *IO-network model*. The full-blown network model is strongly constrained by anatomical and physiological data and is characterized by densely GJ-interconnected (even all-to-all) cells; see a simplified version of the topology in Figure 2(a). It is being actively used in an effort to explain motor function on the scales of motor behavior and adaptation. Among the reasons that we employ such a computational model for our exploration are limiting factors of tissue accessibility, dynamic behavior and observation scale. In our experiments, we will use a simplified version of this network model.

¹The term “compartment” is used for the distinct parts of an accurate white-box neuron representation

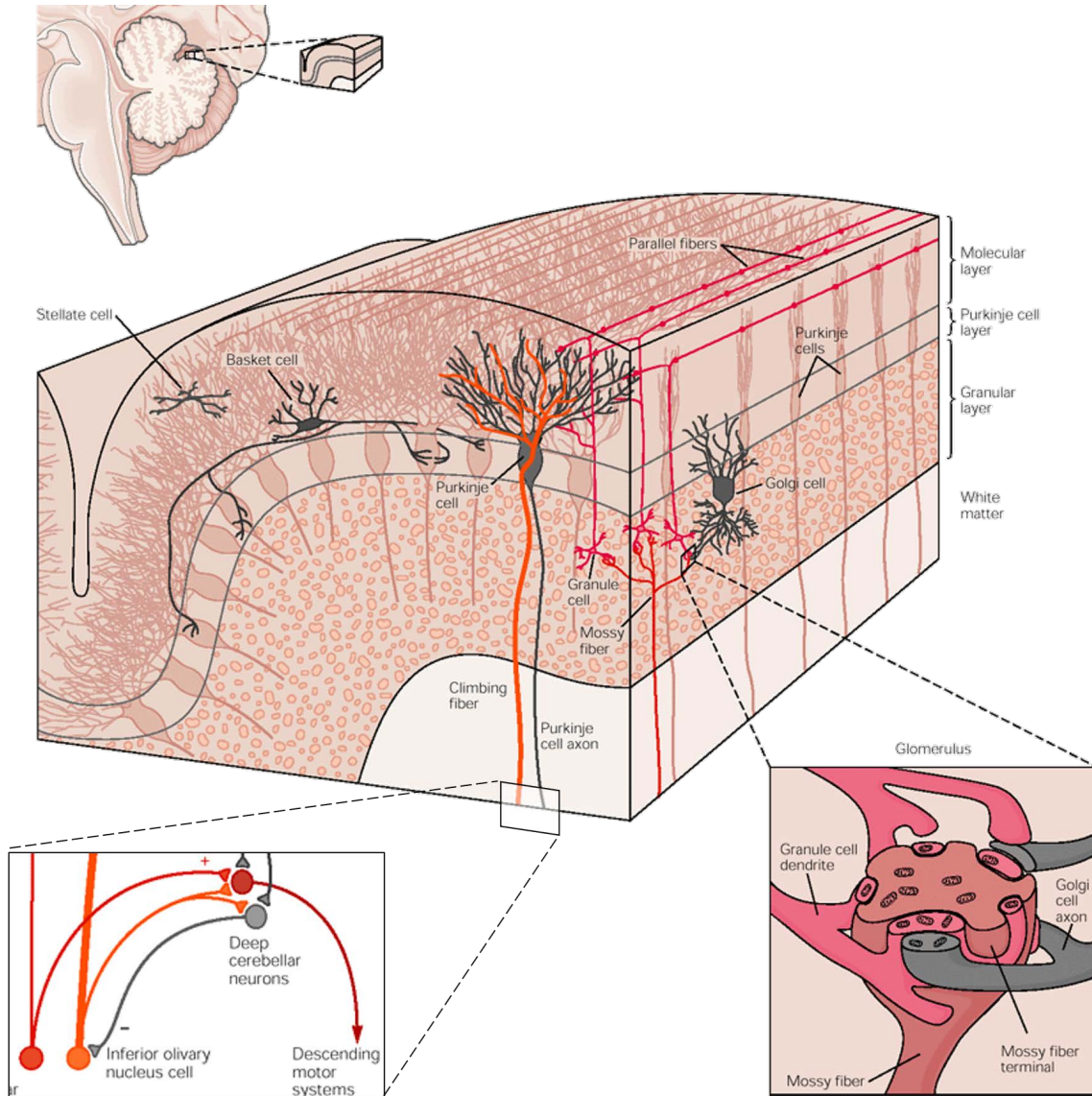


Figure 1: Diagram of the cerebellar anatomy and circuitry. The location and connections between the cerebellum and IO nucleus is visible. [Source: Principles of Neural Science. Kandel]

Inside every IO-cell model, each one of the three compartments includes a number of state variables denoting its electrochemical state and the *neuron state* as a whole. The neuron states are updated at each simulation step; every new state update is based on: i) the neuron state of the previous simulation step of the executed neuron, ii) the previous dendritic states coming from the GJ connectivity, and iii) the externally evoked input to the network, representing the input coming from the rest of the cerebellar circuit.

The somatic, dendritic and axonal compartments of the IO cell are described by the following system of Ordinary Differential Equations (ODEs), respectively, with I_c representing the cumulative GJ influence on the neuron dendrite:

$$\begin{aligned}
 \frac{dV_s}{dt} &= \frac{-(I_{ds} + I_{ls} + I_{Na} + I_{Ca_l} + I_{K_{dr}} + I_h + I_{as}) + I_{app_s}}{C_m} \\
 \frac{dV_d}{dt} &= \frac{-(I_{sd} + I_{ld} + I_{Ca_h} + I_{K_{Ca}} + I_c + I_{as}) + I_{app_d}}{C_m} \\
 \frac{dV_a}{dt} &= \frac{-(I_{Ka} + I_{sa} + I_{la} + I_{Na_a})}{C_m}
 \end{aligned} \tag{1}$$

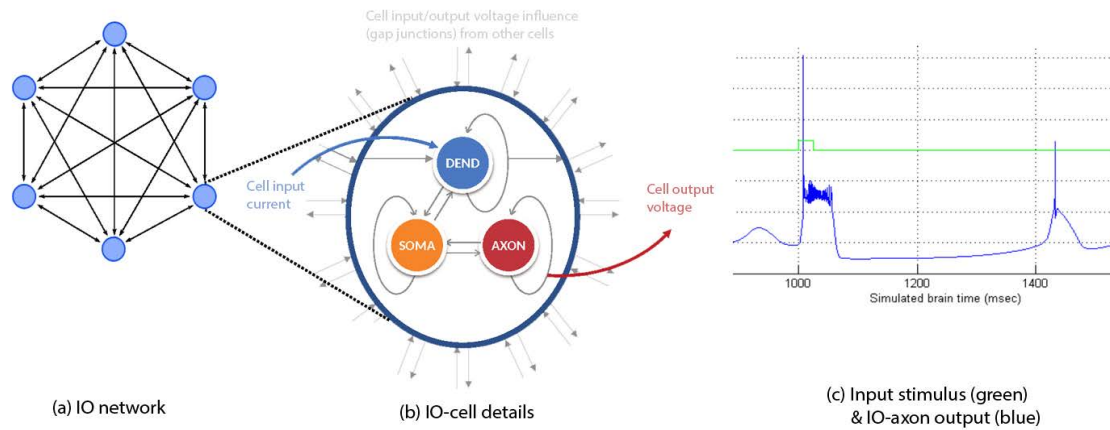


Figure 2: Graphical representation of the IO network model. (a) 8-neuron network; (b) single-neuron model in detail, and (c) sample axon response.

The contribution of various ionic conductances (denoted as currents I_x) to the compartment overall membrane voltage are shown. I_{ds} , I_{sd} , I_{sa} and I_{as} represent dendrite-soma, soma-dendrite, soma-axon and axon-soma interaction currents, respectively. I_{ls} is the soma leakage current, I_{Na} is the Sodium current, I_{Cal} the low-threshold Calcium, I_{Kdr} the slow-Potassium component current, I_h the H -current (also known as the hyperpolarization-activated current). I_{ld} is the dendrite leakage current, I_{CAh} is the high-threshold Calcium current, I_{KCa} is the Calcium-dependent Potassium. Finally, I_{Ka} is the Potassium-axon current, I_{la} is the axon leakage current and I_{Naa} is the Sodium axon current. The GJs are part of the dendritic compartment, thus the compartment receives the extra input coming from the interneuron connection. As you may have deduced from the above, each compartment is effectively modeled as its own HH model (i.e., as a cell itself). Putting together these three HH-modeled compartments yields the whole cell as an *extended HH model* (eHH).

A cell simulator (more accurately, a cell solver) defines multiple *simulation steps* over which the three compartments and the GJ contributions are all evaluated/updated at each step. The step *size* is ultimately defined by the biological features modeled inside the cell and needs to be sufficiently *small* to capture transient phenomena with sufficient details. A typical step size used here is: $\delta = 10 \mu\text{sec}$. This simulation step also defines the *real-time* behavior of the whole network: Every simulation step for the entirety of the network must be completed within (in this case) $10 \mu\text{sec}$ for the execution to be considered real-time. In Figure 2(c), the axon-compartment response of a single cell is shown, when perturbed by a simple pulse in the cell input, i.e., a small I_{app} current applied to the dendrite compartment. Potentially though an applied current can be present in every compartment (I_{appx}).

By solving for multiple cell models over time, we effectively end up with an *IO-network simulator*. Due to solving the problem over and over for a number of time steps, this type of simulator is known as a *transient simulator*. A simplified version of this simulator will be the focus of the final lab exercise for this course.

1.2 The Inferior-Olive challenge

In this final exercise, we will put your newly gained knowledge to the test. You will be given a somewhat simplified, sequential implementation of the IO-network simulator. Your goal, here, will be to perform any and all techniques your learned throughout the course to accelerate the IO code.

Exercise 6.1 (10 points)

By now, it should be clear that simply adding some lines of code to an application may not always result in great speedups. We hope to have taught you that accelerating an application (in Python) is not some magic trick that one performs, but there is an actual strategy behind every technique employed and you – as a programmer with great insight of the underlying hardware – need to carefully think about how to implement each one of them.

Open file `io_model.py` and study the code. Explore the effect of the following parameters in the top of the file and document your observations:

- `sim_seconds`: Simulated brain time in seconds; range: [0.5, ..., 2.0]
- `delta`: Time-step duration in seconds; range: [0.01, 0.03, 0.05]
- `n_cells`: Cell population; range: [1, 2, 10, 30]
- `enable_gapjunctions` Enable or ignore GJ contribution; range: [True, False], with `n_cells = 30`
- `g_CaL`: Low-voltage-activated Ca conductance affecting cell firing; range: [0.7, 1.8, 2.0], by default `g_CaL` is randomized in line 208. You should replace it with these constants and use `n_cells = 1`
- `I_pulse10ms`: 10ms-long square pulse applied to population; range: [2.0, 5.0, 8.0, 10.0]

NB: You do not need to plot different simulation results. You should simply discuss morphological changes that observe in the simulator output.

NB: Do not try to co-vary all parameters simultaneously. Pairwise comparisons are enough.

Exercise 6.2 (90 points)

Accelerate this code using what you have learned throughout the course. You have complete freedom on what combination of techniques to use. Provide the following items in your report:

1. A **pen-and-paper analysis** in addition to the **profiling** of the sequential code.
2. A **plan of attack** before accelerating the program.
3. Your accelerated code **results** for a sweep over
`n_cells = [1, 2, 10, 30, 100, (1000)*,(10000)*,(100000)*]`.
4. **Analysis and profiling** of your accelerated results. Were the findings what you expected?

**(only if it runs fast enough to perform these)*

References

Jornt R. De Gruijl, Paolo Bazzigaluppi, Marcel T. G. de Jeu, and Chris I. De Zeeuw. Climbing fiber burst size and olivary sub-threshold oscillations in a network setting. *PLOS Computational Biology*, 8(12):1–10, 12 2012.

A.L. Hodgkin and A.F. Huxley. A quantitative description of membrane current and its application to conduction and excitation in nerve. *Journal of Physiology*, 117:500–544, 1952.